Chapter 3: Retrieval Models

3.1 Boolean Retrieval Model

The **Boolean Retrieval Model** is one of the earliest and simplest models used in Information Retrieval (IR). It is based on **Boolean logic**, where queries are represented using logical operators such as **AND, OR, and NOT**. The model returns documents as either **relevant or not relevant** — meaning the result is **binary**, without ranking or partial matching.

This model assumes documents can be represented as a set of terms, and a user query is a Boolean expression evaluating whether a document satisfies the condition.

Key Idea of Boolean Model:

A document is retrieved **only if** it satisfies the exact Boolean expression given in the user's query.

Example:
Query → **"Information AND Retrieval"**
Only documents that contain **both** words *information* and *retrieval* will be retrieved.

Components of Boolean Retrieval Model

1. **Term Representation**
   - Each document is represented as a set of keywords (terms).
   - No ranking or weighting of terms is used.
2. **Boolean Operators**
   - Terms are combined using logical operators to form a query.
   - These operators determine document selection.
3. **Exact Matching**
   - A document is either relevant (matches the query) or irrelevant (does not match).

Boolean Operators

The Boolean model uses three main operators:

Operator	Meaning	Example Query	Result
AND	Retrieves documents containing **all** specified terms	computer AND network	Only documents that have both terms
OR	Retrieves documents containing **any** of the terms	AI OR robotics	Documents containing either AI, robotics, or both
NOT	Excludes documents containing a specified term	cloud NOT storage	Documents with "cloud" but not "storage"

Example:

Document Collection:

- **D1:** "Machine learning improves information retrieval"

- **D2:** "Information systems use Boolean logic"
- **D3:** "Machine learning and systems research"
- **D4:** "Boolean retrieval model supports AND OR and NOT operations"

Query 1 → **Machine AND learning**
Result → **D1, D3**

Query 2 → **Boolean OR machine**
Result → **D1, D3, D4**

Query 3 → **Information AND NOT machine**
Result → **D2**

Query Processing in Boolean Model

Boolean query processing involves the following steps:

1. **Tokenization and Preprocessing**
    - Documents are tokenized and normalized (case folding, stopword removal).
2. **Mapping Tokens to Postings Lists**
    - The inverted index is used to identify documents containing each term.
3. **Applying Boolean Operators**
    - Operators are applied to postings lists using set operations:
        - **AND → intersection**
        - **OR → union**
        - **NOT → set difference**
4. **Result Generation**
    - Only documents satisfying the Boolean condition are returned.

Example of Set Operations:

Postings Lists:

- learning → {D1, D3}
- machine → {D1, D3}
- Boolean → {D2, D4}

Query: `(machine AND learning) OR Boolean`

Step-by-step:

- machine AND learning → {D1, D3}
- {D1, D3} OR {D2, D4} → **{D1, D2, D3, D4}**

Final Result → D1, D2, D3, D4

Advantages of Boolean Model

1. **Simple and Easy to Understand**
    - Users familiar with basic logic find it intuitive.

2. **Exact Control Over Query**
    - Useful for professional searchers (law, medicine, library systems).
3. **Efficient with Inverted Index**
    - Boolean matching can be implemented quickly.
4. **Deterministic Results**
    - No ranking ambiguity — documents are simply relevant or not.

Limitations of Boolean Model

1. **Binary Result — No Ranking**
    - All matching documents are treated equally, leading to poor user experience when result sets are large.
2. **Requires Precise Queries**
    - Users must think carefully about query structure.
3. **No Concept of Partial Matching**
    - A document containing *almost related content* may not be retrieved.
4. **Large Result or Zero Result Problem**
    - A query may retrieve too many documents (OR) or none at all (strict AND).

Use Cases of Boolean Model

- Digital libraries
- Legal and patent search systems
- Structured archival databases
- Metadata-based retrieval systems

Summary

The **Boolean Retrieval Model** is a foundational model in Information Retrieval based on logical operators. It provides **exact match retrieval**, making it useful for structured and professional search environments. However, since it does not support ranking or partial matching, modern systems often combine Boolean retrieval with more advanced models such as the **Vector Space Model** and **Probabilistic Retrieval Models**.

3.2 Vector Space Model (VSM)

The **Vector Space Model (VSM)** is one of the most widely used retrieval models in modern Information Retrieval systems. Unlike the Boolean Model, which provides only binary results (relevant or not relevant), the Vector Space Model supports **ranked retrieval**. This means documents are scored based on how similar they are to the user query, and results are presented in a ranked order from most relevant to least relevant.

The basic idea of VSM is to represent both documents and queries as vectors in a multi-dimensional space where each dimension corresponds to a unique term in the collection.

Key Concepts in VSM

1. **Documents as Vectors**
   - Each document is represented as a vector of numeric weights corresponding to terms.

   Example: D=(w1,w2,w3,...,wn)

2. **Query as a Vector**
   - The user query is also represented as a vector using the same weighting process.
3. **Similarity Measurement**
   - The similarity between the query vector and document vectors is calculated using a similarity function, most commonly **Cosine Similarity**.

Term Weighting: TF-IDF Method

To measure importance of terms in documents, VSM uses **TF-IDF weighting**, which stands for:

✓ TF → Term Frequency
✓ IDF → Inverse Document Frequency

Together, TF-IDF gives higher weight to terms that appear frequently in a document but are rare in the entire collection.

(1) Term Frequency (TF)

TF measures how often a word occurs in a document.

$$TF = \frac{\text{Number of times term appears in document}}{\text{Total number of terms in document}}$$

Example:

If the term *information* appears **3 times** in a document with **100 words**:

$$TF = \frac{3}{100} = 0.03$$

(2) Inverse Document Frequency (IDF)

IDF measures how important a term is across the whole document collection.

$$IDF = \log\left(\frac{N}{df}\right)$$

Where:

- **N = total number of documents**
- **df = number of documents containing the term**

Example:

If *retrieval* appears in **2 out of 10 documents**:

$$IDF = \log\left(\frac{10}{2}\right) = \log(5)$$

(3) TF-IDF Weight

$$TF\text{–}IDF = TF \times IDF$$

This value represents the importance of a term in a specific document.

Cosine Similarity: Query–Document Matching

Once TF-IDF weights are computed, both documents and query are transformed into vectors. The similarity between them is calculated using **Cosine Similarity**, which measures the angle between two vectors.

$$\text{Cosine Similarity} = \frac{\vec{D} \cdot \vec{Q}}{\left\|\vec{D}\right\| \times \left\|\vec{Q}\right\|}$$

Where:

- $\vec{D} \cdot \vec{Q}$ = dot product of vectors
- $\left\|\vec{D}\right\|$ and $\left\|\vec{Q}\right\|$ = magnitudes

Similarity values range from **0 to 1**:

- **1 → perfect match**
- **0 → no similarity**

Example of Vector Space Ranking

Document Collection:

Document	Text
D1	"Machine learning improves information retrieval"
D2	"Information systems use learning models"

Query → **"learning information"**

Step-by-step:

1. Identify terms: {learning, information}
2. Compute TF-IDF weights
3. Form document vectors
4. Compute cosine similarity

Final Result (Example Ranking):

Document	Similarity Score	Rank
D1	0.91	1
D2	0.62	2

So, **D1 is more relevant** to the query.

Advantages of Vector Space Model

1. **Supports Ranked Retrieval**
   - Most relevant documents appear first.
2. **Partial Matching Allowed**
   - Documents do not need to match all query terms.
3. **Term Weighting Improves Accuracy**

    - TF-IDF gives importance to meaningful terms.
4. **Mathematically Strong and Scalable**
    - Works well for medium and large document collections.

Limitations of Vector Space Model

1. **High Dimensionality**
    - Each unique term forms a dimension leading to large vectors.
2. **Computationally Expensive**
    - Requires matrix operations and similarity calculations.
3. **Assumes Independence Between Terms**
    - Does not understand synonyms or semantic meaning.
4. **Query Representation May Not Fully Capture User Intent**
    - Relies only on statistical term matching.

Applications of VSM

- Web search engines
- Text classification and clustering
- Recommendation systems
- Email spam filtering
- Semantic search platforms

Summary

The **Vector Space Model** is a foundational IR model that uses **TF-IDF weighting** and **cosine similarity** to score and rank documents based on their similarity to a user query. It overcomes limitations of the Boolean model by supporting **partial matching and ranked retrieval**, making it highly effective for real-world applications such as search engines and document analysis.

3.3 Probabilistic Retrieval Model

The **Probabilistic Retrieval Model** is based on estimating the likelihood that a given document is relevant to a user's query. Instead of binary matching (like the Boolean Model) or similarity matching (like the Vector Space Model), this model uses probability theory to determine how likely a document is to satisfy the user's information need.

The fundamental idea is:

Documents should be ranked according to the probability of their relevance to the query.
The higher the probability, the higher the rank.

Core Principle: Probability of Relevance

For any document D and query Q, the model attempts to compute:

$$P(R|D,Q)$$

Where:

- R = relevance
- P(R|D, Q) = probability that document D is relevant to query Q

Documents are ranked in descending order of this probability.

Binary Independence Model (BIM)

The **Binary Independence Model** is the most widely used form of the probabilistic retrieval model. It assumes:

✓ Terms are independent of each other
✓ A term is either present or absent (binary, no frequency consideration)

Based on these assumptions, BIM calculates the probability using statistical information from relevant and non-relevant documents.

Formula for BIM Ranking

The ranking score for a document is calculated using:

$$\text{Score}(D) = \sum_{t \in Q} \log \left(\frac{p_t(1-q_t)}{(1-p_t)q_t} \right)$$

Where:

- ptp_tpt = probability term ttt appears in relevant documents
- qtq_tqt = probability term ttt appears in non-relevant documents

Documents with higher scores are considered more relevant.

Example:

Suppose the query contains terms:
→ **{database, indexing}**

Data collected:

Term	Appears in relevant docs (%)	Appears in non-relevant docs (%)
database	80%	20%
indexing	60%	10%

Using the formula, each document gets a computed relevance probability.

A document containing *both* terms will score higher than one containing only one term — and both will score higher than a document containing none.

Relevance Feedback

A key strength of probabilistic models is that they allow **relevance feedback**.
After initial retrieval, the user marks some documents as relevant or non-relevant. The system uses this information to update the probabilities and improve ranking.

Two types:

1. **Explicit Relevance Feedback**
    - User manually marks documents as relevant or irrelevant.
    - Helps refine query terms and update probability estimates.
2. **Implicit Relevance Feedback**
    - System observes user behavior (clicks, reading time, scrolling, etc.)
    - No manual marking required.\

Rocchio Algorithm (Application in Feedback)

A common method used along with feedback updates the query by:

$$Q_{\text{new}} = \alpha Q + \beta \frac{1}{|R|} \sum D_r - \gamma \frac{1}{|NR|} \sum D_{nr}$$

Where:

- R = relevant documents
- NR = non-relevant documents
- α, β, γ = tuning parameters

This improves future retrieval quality by refining the query vector.

Bayesian Retrieval Approach

The probabilistic model is strongly connected to **Bayes' Theorem**:

$$P(R|D,Q) = \frac{P(D|R,Q) \cdot P(R)}{P(D)}$$

Where:

- $P(R)$ is the prior probability of relevance
- $P(D|R,Q)$ indicates how likely the document's terms appear in relevant documents

Bayesian reasoning helps predict relevance based on past user interactions or known relevance patterns.

Advantages of Probabilistic Model

1. **Provides Ranked Output**
   - More relevant documents appear first.
2. **Data-Driven Approach**
   - Uses statistical information to improve results.
3. **Learns from User Feedback**
   - Continuously improves retrieval accuracy.
4. **More realistic than binary matching models**
   - Captures uncertainty and user intention better.

Limitations of Probabilistic Model

1. **Initial Uncertainty**
   - Assumes probabilities without knowing relevance initially.
2. **Independence Assumption**
   - Assumes terms are independent — which is not realistic in natural language.

3. **Computational Complexity**
    - Probability estimation and feedback processing require intensive computation.
4. **Depends on User Input**
    - Relevance feedback works only if users cooperate.

Applications of Probabilistic Retrieval

- Web search ranking algorithms
- Online recommendation engines
- Personalized search systems
- Machine learning-based text mining tools

Summary

The **Probabilistic Retrieval Model** ranks documents based on the probability of their relevance to the user query. Using statistical measures such as term probability, Bayesian inference, and relevance feedback, the model improves retrieval performance over time. It is more flexible and adaptive than Boolean and Vector Space Models and strongly influences modern search systems like Google and Bing.

Comparative Summary of Retrieval Models

Feature	Boolean Model	Vector Space Model (VSM)	Probabilistic Model
Matching Type	Exact match (binary)	Partial matching	Probabilistic matching
Output Type	Relevant / Not relevant (no ranking)	Ranked list based on similarity score	Ranked list based on probability of relevance
Use of Term Frequency	No	Yes (TF-IDF)	Sometimes (in advanced versions)
Use of Document Frequency	No	Yes (IDF)	Yes (probability estimation)
Query Structure	Boolean expressions with AND/OR/NOT	Treated as a vector	Treated as a probabilistic evidence set
User Skill Requirement	High (must know Boolean logic)	Low to Medium	Medium
Handling Synonyms	Poor	Moderate	Good (with feedback)
Relevance Feedback	Not supported	Supported (Rocchio)	Strongly supported
Best For	Structured databases, legal, library search	Search engines, ranking tasks	Personalized search, adaptive systems
Strength	Simple and deterministic	Ranked retrieval, scalable	Learns and improves with user feedback
Limitation	No ranking, strict matching	High dimensionality and computational cost	Requires relevance estimates and user interaction